



UNIVERSIDAD CATÓLICA DE SALTA
Licenciatura en Ciencia de Datos

Trabajo Práctico N° 3

Análisis de algoritmos computacionales

Grupo 5

Integrantes:

Baldoma, Fatima
Fagnani, Gianella Maria
Luna, Facundo Gaston
Marquetti, Marcelo Sergio
Semino Giaccone, Esteban David
Vidal Bernal, America

Profesor: Ing. Guillermo Saravia Sanchez

Fecha: 8 de octubre de 2026

Índice

Aclaraciones previas	3
Introducción	3
1. Eficiencia de un algoritmo	4
2. Fases en el análisis de la eficiencia	5
2.1. Análisis a priori (teórico)	5
2.2. Análisis a posteriori (empírico o experimental)	5
3. Tamaño o talla de un problema	6
4. Métodos para medir el tiempo de ejecución	7
4.1. Método teórico: conteo de operaciones elementales	7
4.2. Método empírico: medición de tiempos de ejecución	8
4.3. Método intermedio: conteo de la operación crítica por instrumentación	8
5. Complejidad de un algoritmo y órdenes de complejidad	9
5.1. Concepto de complejidad	9
5.2. Órdenes de complejidad	9
6. Notación asintótica	11
6.1. Notación O (O grande): cota superior	11
6.2. Notación Ω (Omega): cota inferior	11
6.3. Notación Θ (Theta): cota ajustada	11
6.4. Para qué se utiliza	12
7. Determinación del orden de complejidad	13
7.1. a) Condicional con bucles anidados	13
7.2. b) Función con bucles secuenciales	15
8. Resolución de algoritmos	17
8.1. a) Perímetro de una circunferencia con validación del radio	17

8.2. b) Seno y coseno de un ángulo en grados	18
8.3. c) Hipotenusa de un triángulo rectángulo	19
8.4. d) Cara de un dado mostrada en letras	20
8.5. e) Sumatoria de 10 números generados al azar	21
8.6. f) Promedio de edades en una veterinaria	22
8.7. g) Factorial de un número	24
Conclusión	25

Aclaraciones previas

La primera parte de este trabajo (puntos 1 a 7) desarrolla los conceptos teóricos de la Unidad III referidos al análisis de la eficiencia de algoritmos. La segunda parte (punto 8) presenta la resolución de siete algoritmos en pseudocódigo, utilizando la sintaxis del software PSeInt, manteniendo el criterio adoptado en el Trabajo Práctico N° 1: centrar el análisis en la lógica de resolución, evitando que la sintaxis de un lenguaje particular interfiera en la evaluación del razonamiento algorítmico.

El código fuente completo de cada ejercicio se encuentra disponible en el siguiente repositorio de GitHub, con el fin de facilitar su verificación y prueba en el entorno de PSeInt. Cada ejercicio corresponde a un archivo `.psc` independiente:

<https://github.com/Facug1/data-science-coursework/tree/main/primer-anio/analisis-algoritmos/tp-3/ejercicios>

Introducción

Un mismo problema admite, en general, infinitas soluciones algorítmicas correctas. Lo que distingue a unas de otras es la cantidad de recursos que consumen: tiempo de procesamiento y espacio de memoria. El análisis de la eficiencia proporciona las herramientas formales para comparar esas soluciones de manera objetiva e independiente del equipo donde se ejecuten, permitiendo anticipar el comportamiento de un algoritmo antes de implementarlo y predecir cómo escalará cuando el volumen de datos crezca.

1. Eficiencia de un algoritmo

Definir eficiencia de un algoritmo.

La **eficiencia** de un algoritmo es la medida de la cantidad de recursos computacionales que éste consume para resolver un problema de un tamaño determinado. Un algoritmo es más eficiente que otro cuando, resolviendo el mismo problema y produciendo el mismo resultado correcto, requiere menos recursos.

Los dos recursos que se analizan son:

- **Eficiencia temporal (complejidad en tiempo):** el tiempo que el algoritmo tarda en producir la salida. Se mide en cantidad de operaciones elementales ejecutadas, no en segundos, para independizar el análisis del hardware.
- **Eficiencia espacial (complejidad en espacio):** la cantidad de memoria que el algoritmo necesita durante su ejecución, por encima de la ocupada por los datos de entrada.

Es importante señalar que la eficiencia es una propiedad *relativa*: sólo tiene sentido al comparar algoritmos que resuelven el mismo problema. Además, suele existir un compromiso (*trade-off*) entre ambos recursos: con frecuencia un algoritmo puede reducir su tiempo de ejecución a costa de consumir más memoria, y viceversa.

Finalmente, la eficiencia es una cualidad distinta de la corrección. Un algoritmo debe ser, ante todo, correcto; la eficiencia es el criterio que permite elegir entre varias soluciones correctas.

[↑ Volver al índice](#)

2. Fases en el análisis de la eficiencia

¿Cuáles son las dos fases en el análisis de la eficiencia de algoritmos?

El análisis de la eficiencia se realiza en dos fases sucesivas y complementarias:

2.1. Análisis a priori (teórico)

Se efectúa **antes** de ejecutar el algoritmo, trabajando sobre su descripción (pseudocódigo o código fuente). Consiste en obtener una función matemática $T(n)$ que acote la cantidad de operaciones elementales ejecutadas en función del tamaño n de la entrada.

Sus características son:

- Es **independiente** del lenguaje de programación, del compilador y de la máquina donde se ejecute.
- No requiere implementar el algoritmo.
- El resultado se expresa mediante notación asintótica, por ejemplo $O(n^2)$.

2.2. Análisis a posteriori (empírico o experimental)

Se realiza **después** de implementar el algoritmo, ejecutándolo con juegos de datos reales y midiendo los tiempos y consumos obtenidos.

Sus características son:

- **Depende** del hardware, del sistema operativo, del lenguaje y del compilador utilizados.
- Permite verificar experimentalmente las predicciones del análisis a priori.
- Revela el peso de las constantes ocultas que la notación asintótica descarta, factor decisivo cuando se comparan algoritmos del mismo orden de complejidad.

Ambas fases son complementarias: el análisis a priori indica cómo escalará el algoritmo cuando el volumen de datos crezca, mientras que el análisis a posteriori informa cuánto cuesta realmente ejecutarlo en un entorno concreto.

↑ Volver al índice

3. Tamaño o talla de un problema

¿Cuál es el tamaño o talla de un problema?

El **tamaño** o **talla** de un problema, habitualmente denotado con la letra n , es el parámetro que cuantifica el volumen de los datos de entrada y respecto del cual se expresa el tiempo de ejecución del algoritmo.

La elección de n no es arbitraria: debe ser la magnitud de la entrada que realmente determina el trabajo que el algoritmo debe realizar. Según el problema, la talla puede ser:

- La **cantidad de elementos** de un arreglo o lista, en algoritmos de búsqueda y ordenamiento.
- El **valor** de un número entero, en algoritmos como el cálculo del factorial o la determinación de primalidad.
- La **cantidad de dígitos o bits** de un número, en algoritmos aritméticos sobre enteros grandes.
- La **cantidad de vértices y aristas** de un grafo, en algoritmos de recorrido o camino mínimo.
- Las **dimensiones** de una matriz, por ejemplo $n \times m$.

Un problema puede requerir más de un parámetro de talla. En la multiplicación de una matriz de $n \times m$ por otra de $m \times p$, el tiempo depende de los tres valores y la función de costo se expresa como $T(n, m, p)$.

Definir correctamente la talla es el primer paso de todo análisis, porque la función $T(n)$ y el orden de complejidad resultante carecen de sentido sin indicar respecto de qué magnitud se están midiendo.

[↑ Volver al índice](#)

4. Métodos para medir el tiempo de ejecución

Explicar al menos 2 métodos para medir el tiempo de ejecución en un algoritmo.

4.1. Método teórico: conteo de operaciones elementales

Consiste en determinar analíticamente, a partir del pseudocódigo, cuántas operaciones elementales ejecuta el algoritmo en función de la talla n . Se considera **operación elemental** a toda instrucción cuyo tiempo de ejecución está acotado por una constante independiente de n : una asignación, una comparación, una operación aritmética, el acceso a una posición de un arreglo o la escritura de un valor.

El procedimiento se apoya en tres reglas:

- **Instrucciones simples:** cada una cuenta como 1, tiempo constante $O(1)$.
- **Regla de la suma (secuencia):** si dos bloques consecutivos cuestan $T_1(n)$ y $T_2(n)$, el bloque completo cuesta $T_1(n) + T_2(n)$, cuyo orden es $O(\max(T_1(n), T_2(n)))$.
- **Regla del producto (anidamiento):** si un bucle que itera $f(n)$ veces contiene un cuerpo de costo $g(n)$, el costo total es $f(n) \cdot g(n)$.

A las estructuras condicionales se les asigna, en el análisis del peor caso, el costo de la rama más costosa más el de evaluar la condición.

Ejemplo. Para el siguiente fragmento:

Instrucción	Veces que se ejecuta
<code>suma <- 0</code>	1
<code>Para i <- 1 Hasta n</code>	$n + 1$ (incluye la comparación final)
<code> suma <- suma + i</code>	n

El total es $T(n) = 1 + (n + 1) + n = 2n + 2$, que pertenece a $O(n)$.

Ventajas: es independiente de la máquina, del lenguaje y de los datos concretos, y permite predecir el comportamiento para cualquier valor de n , incluso para tallas que sería imposible ejecutar.

Limitaciones: ignora las constantes multiplicativas, que en la práctica pueden ser determinantes, y su cálculo resulta complejo en algoritmos recursivos o con muchas ramificaciones.

4.2. Método empírico: medición de tiempos de ejecución

Consiste en implementar el algoritmo y cronometrar su ejecución real sobre distintos juegos de datos, registrando el instante inicial y el final mediante los relojes que provee el sistema (por ejemplo `time.perf_counter()` en Python o `System.nanoTime()` en Java).

El procedimiento habitual es:

1. Ejecutar el algoritmo con entradas de talla creciente, por ejemplo $n = 100, 1,000, 10,000, 100,000$.
2. Repetir cada medición varias veces y promediar, para atenuar el ruido introducido por otros procesos del sistema operativo.
3. Tabular y graficar los tiempos obtenidos en función de n , verificando si la curva resultante se ajusta a la forma prevista por el análisis teórico.

Ventajas: refleja el costo real en un entorno concreto, incorpora el efecto de las constantes ocultas y permite discriminar entre algoritmos del mismo orden asintótico.

Limitaciones: los resultados sólo son válidos para el hardware, el lenguaje y los datos utilizados; exige implementar el algoritmo; y no permite extrapolar con seguridad a tallas mucho mayores que las ensayadas.

4.3. Método intermedio: conteo de la operación crítica por instrumentación

Una alternativa que combina ambos enfoques consiste en instrumentar el código con un contador que se incrementa cada vez que se ejecuta la **operación crítica** o característica del algoritmo —la comparación entre elementos en un ordenamiento, por ejemplo— y ejecutarlo con entradas de talla creciente.

Este método conserva la independencia del hardware propia del análisis teórico, porque cuenta operaciones y no segundos, pero obtiene los valores de manera experimental, lo que evita la dificultad del cálculo analítico en algoritmos complejos.

[↑ Volver al índice](#)

5. Complejidad de un algoritmo y órdenes de complejidad

¿Qué es la complejidad de un algoritmo? ¿Cuáles son los órdenes de complejidad?

5.1. Concepto de complejidad

La **complejidad** de un algoritmo es la función que describe cómo crece el consumo de recursos a medida que aumenta la talla n del problema. No interesa el valor exacto de $T(n)$ sino su **tasa de crecimiento**, es decir, la forma en que la curva se comporta cuando n tiende a infinito.

Por eso, al expresar la complejidad se descartan:

- Las **constantes multiplicativas**: $5n$ y $100n$ pertenecen ambas a $O(n)$.
- Los **términos de menor orden**: en $3n^2 + 7n + 12$ sólo subsiste n^2 , porque para valores grandes de n domina sobre los demás.

Dado que el costo puede variar con la disposición concreta de los datos, se distinguen tres casos:

- **Mejor caso**: la entrada más favorable. En una búsqueda lineal, que el elemento buscado esté en la primera posición: $\Omega(1)$.
- **Peor caso**: la entrada más desfavorable. En la misma búsqueda, que el elemento esté al final o no esté: $O(n)$. Es el caso más utilizado, porque ofrece una garantía: el algoritmo nunca tardará más que eso.
- **Caso promedio**: el valor esperado sobre todas las entradas posibles, ponderado por su probabilidad. Es el más realista, pero también el más difícil de calcular.

5.2. Órdenes de complejidad

Los órdenes más frecuentes, de menor a mayor costo, son:

Orden	Denominación	Ejemplo típico
$O(1)$	Constante	Acceder a un elemento de un arreglo por su índice
$O(\log n)$	Logarítmica	Búsqueda binaria en un arreglo ordenado
$O(n)$	Lineal	Búsqueda secuencial; recorrer una lista
$O(n \log n)$	Cuasi-lineal	Mergesort, Quicksort (caso promedio), Heapsort
$O(n^2)$	Cuadrática	Ordenamiento burbuja, selección, inserción
$O(n^3)$	Cúbica	Multiplicación clásica de matrices $n \times n$
$O(n^k)$	Polinómica	Algoritmos con k bucles anidados
$O(2^n)$	Exponencial	Subconjuntos de un conjunto; Fibonacci recursivo ingenuo
$O(n!)$	Factorial	Permutaciones; fuerza bruta del viajante de comercio

La relación de dominancia entre ellos es:

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

Habitualmente se agrupan en dos grandes familias:

- **Algoritmos tratables:** los de complejidad polinómica —hasta $O(n^k)$ —, considerados computacionalmente viables.
- **Algoritmos intratables:** los de complejidad exponencial o factorial, cuyo tiempo de ejecución se vuelve prohibitivo incluso para tallas moderadas.

La diferencia práctica es enorme. Suponiendo una máquina que ejecuta 10^6 operaciones por segundo y una talla $n = 50$: un algoritmo $O(n^2)$ termina en microsegundos, mientras que uno $O(2^n)$ requeriría aproximadamente 35 años.

↑ Volver al índice

6. Notación asintótica

Explicar notación asintótica y para qué se utiliza.

La **notación asintótica** es el conjunto de herramientas matemáticas que permite describir el comportamiento límite de una función de costo cuando la talla del problema tiende a infinito, prescindiendo de los detalles que no afectan esa tendencia.

El término proviene del concepto matemático de *asíntota*: así como una curva se aproxima indefinidamente a una recta sin necesariamente alcanzarla, la función $T(n)$ se aproxima a un cierto patrón de crecimiento para valores grandes de n .

6.1. Notación O (O grande): cota superior

Se dice que $f(n) \in O(g(n))$ si existen constantes positivas c y n_0 tales que:

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{para todo } n \geq n_0$$

Significa que f no crece más rápido que g . Es la notación más utilizada, porque expresa una **garantía sobre el peor caso**: el algoritmo nunca tardará más que el orden indicado.

Ejemplo. $3n^2 + 7n + 12 \in O(n^2)$, tomando $c = 22$ y $n_0 = 1$.

6.2. Notación Ω (Omega): cota inferior

Se dice que $f(n) \in \Omega(g(n))$ si existen constantes positivas c y n_0 tales que:

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{para todo } n \geq n_0$$

Significa que f crece al menos tan rápido como g . Describe el **mejor caso** y establece un piso: el algoritmo no puede ser más rápido que ese orden.

6.3. Notación Θ (Theta): cota ajustada

Se dice que $f(n) \in \Theta(g(n))$ si $f(n) \in O(g(n))$ y $f(n) \in \Omega(g(n))$ simultáneamente, es decir, si existen constantes positivas c_1 , c_2 y n_0 tales que:

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{para todo } n \geq n_0$$

Es la caracterización más precisa, porque indica que la función crece *exactamente* al ritmo de g . Sólo puede emplearse cuando el mejor y el peor caso coinciden en orden.

6.4. Para qué se utiliza

La notación asintótica se emplea para:

- **Comparar algoritmos objetivamente**, con independencia del lenguaje, el compilador y la máquina en que se ejecuten.
- **Predecir la escalabilidad**: si se duplica la talla de la entrada, un algoritmo $O(n)$ duplica su tiempo mientras que uno $O(n^2)$ lo cuadruplica.
- **Simplificar el análisis**, descartando constantes y términos de menor orden que no alteran la tendencia de crecimiento.
- **Fundamentar decisiones de diseño**, permitiendo elegir la estructura de datos o la estrategia adecuada antes de escribir una sola línea de código.

Advertencia. El análisis asintótico es válido para valores grandes de n . Para tallas pequeñas, un algoritmo de orden superior pero con constantes bajas puede resultar más rápido en la práctica: por eso varias implementaciones reales de Quicksort conmutan a ordenamiento por inserción cuando el subarreglo tiene menos de una decena de elementos.

[↑ Volver al índice](#)

7. Determinación del orden de complejidad

Determinar el orden de complejidad de los siguientes algoritmos, justificando su elección.

7.1. a) Condicional con bucles anidados

```
1 a = 0
2 b = int(input("ingrese valor b: "))
3 if b == 100:
4     for i in range(0, n, 1):
5         for j in range(0, n, 1):
6             a += 1
7     print("valor a entrando con b=100: ", a)
8 else:
9     for i in range(0, n, 1):
10        a += 1
11    print("valor a entrando con b<>100: ", a)
```

Algoritmo 1: Algoritmo con bucles dependientes de una condición

Análisis

El algoritmo está estructurado como un condicional cuyas dos ramas tienen costos muy distintos. La talla del problema es n , la variable que gobierna los bucles. El valor de b , en cambio, es un dato de entrada que no depende de n y sólo determina qué rama se ejecuta.

Costo fijo previo. Las dos primeras líneas —la asignación $a=0$ y la lectura de b — y la evaluación de la condición $\text{if } b==100$ son operaciones elementales de costo constante $O(1)$.

Rama verdadera ($b = 100$). Se ejecutan dos bucles anidados. El bucle externo realiza n iteraciones y, para cada una de ellas, el bucle interno realiza otras n iteraciones con un cuerpo de costo constante ($a+=1$). Por la regla del producto:

$$T_{\text{si}}(n) = n \cdot n \cdot O(1) = n^2 \in O(n^2)$$

Rama falsa ($b \neq 100$). Se ejecuta un único bucle de n iteraciones con cuerpo de costo constante:

$$T_{\text{sino}}(n) = n \cdot O(1) = n \in O(n)$$

Conclusión

Dado que el análisis de complejidad se expresa convencionalmente en términos del peor caso, y que la rama más costosa es la que se ejecuta cuando $b = 100$:

$$T(n) \in O(n^2) \quad \text{— complejidad cuadrática}$$

De manera más precisa, el comportamiento puede caracterizarse en los tres casos:

Caso	Condición	Orden
Mejor caso	$b \neq 100$	$\Omega(n)$
Peor caso	$b = 100$	$O(n^2)$

No es posible dar una cota ajustada Θ para el algoritmo completo, porque el mejor y el peor caso pertenecen a órdenes distintos. Sí puede darse para cada rama por separado: $\Theta(n^2)$ para la primera y $\Theta(n)$ para la segunda.

Observación sobre el código. Cabe señalar que el fragmento presenta un error de implementación: la variable `n` que controla los bucles nunca es definida ni leída, por lo que su ejecución produciría un `NameError`. El análisis se realiza asumiendo que `n` contiene la talla del problema, que es la intención evidente del enunciado.

7.2. b) Función con bucles secuenciales

```
1 // Pseudocódigo
2 Funcion ejemplo (n: entero): entero
3 Empieza
4     Variables a, j, k enteros
5     Para j desde 1 hasta n hacer
6         a = a + j
7     Fin para
8     Para k desde n hasta 1 hacer
9         a = a - 1
10        a = a * 2
11    Fin para
12    Devolver a
13 Termina
```

Algoritmo 2: Función con dos bucles consecutivos

Análisis

A diferencia del caso anterior, aquí los dos bucles son **consecutivos**, no anidados. La talla del problema es n , el parámetro de la función.

Primer bucle. La variable j recorre los valores desde 1 hasta n , realizando n iteraciones. El cuerpo contiene una única operación elemental ($a=a+j$), de costo constante:

$$T_1(n) = n \cdot O(1) = n$$

Segundo bucle. La variable k recorre los valores desde n hasta 1 en orden descendente, realizando también n iteraciones. El cuerpo contiene dos operaciones elementales ($a=a-1$ y $a=a*2$), igualmente de costo constante:

$$T_2(n) = n \cdot 2 \cdot O(1) = 2n$$

Declaraciones y retorno. La declaración de variables y la instrucción `Devolver a` aportan un costo constante c .

Costo total

Por la regla de la suma, los bloques consecutivos se adicionan:

$$T(n) = T_1(n) + T_2(n) + c = n + 2n + c = 3n + c$$

Descartando la constante aditiva c y el factor multiplicativo 3, que no alteran la tasa de crecimiento:

$$T(n) \in \Theta(n) \quad \text{— complejidad lineal}$$

En este caso sí corresponde emplear la notación Θ , porque ambos bucles se ejecutan siempre la misma cantidad de veces, independientemente de los valores que tomen los datos. No existen ramificaciones, y por lo tanto el mejor caso, el peor caso y el caso promedio coinciden: $T(n) \in O(n)$ y $T(n) \in \Omega(n)$.

Observación clave. El punto central de este ejercicio es distinguir el **anidamiento** de la **secuencia**. Si el segundo bucle estuviese contenido dentro del primero, se aplicaría la regla del producto y el orden resultaría $O(n^2)$. Al estar uno a continuación del otro, se aplica la regla de la suma y el orden se mantiene lineal, $O(\max(n, n)) = O(n)$.

Observación sobre el código. El pseudocódigo utiliza la variable `a` sin inicializarla previamente, lo que constituye un error de implementación. Dicha omisión no afecta al análisis de complejidad, dado que una asignación inicial agregaría únicamente una operación de costo constante.

[↑ Volver al índice](#)

8. Resolución de algoritmos

Resolver los siguientes algoritmos.

8.1. a) Perímetro de una circunferencia con validación del radio

Mostrar el perímetro de una circunferencia, siempre y cuando el radio que se ingresa sea mayor a cero (controlar dicho ingreso).

El perímetro se calcula mediante la fórmula $P = 2\pi r$. La validación del radio se implementa con una estructura Repetir...Hasta Que, que garantiza al menos una lectura y vuelve a solicitar el dato mientras el valor ingresado no sea válido.

```
1 Algoritmo PerimetroCircunferencia
2   Definir radio, perimetro Como Real
3
4   Repetir
5       Escribir "Ingrese el radio de la circunferencia (debe
        ser mayor a 0):"
6       Leer radio
7       Si radio <= 0 Entonces
8           Escribir "Error: el radio debe ser mayor a 0.
            Intente nuevamente."
9       FinSi
10      Hasta Que radio > 0
11
12      perimetro <- 2 * PI * radio
13
14      Escribir "El perimetro de la circunferencia es: ", perimetro
15
16 FinAlgoritmo
```

Algoritmo 3: Perímetro de una circunferencia con validación

Complejidad: $O(1)$ en cantidad de operaciones por cada ingreso válido. El bucle de validación depende de la cantidad de intentos fallidos del usuario, no de la talla del problema.

[↑ Volver al índice](#)

8.2. b) Seno y coseno de un ángulo en grados

Se lee desde el teclado el valor de un ángulo en grados. Se desea mostrar el coseno y el seno.

Las funciones trigonométricas de PSeInt —al igual que las de la mayoría de los lenguajes de programación— esperan el argumento expresado en **radianes**. Por lo tanto, es necesario convertir previamente el valor ingresado aplicando:

$$\text{radianes} = \text{grados} \cdot \frac{\pi}{180}$$

Omitir esta conversión constituye el error más frecuente en este tipo de ejercicios: el programa no fallaría, pero devolvería valores incorrectos.

```
1 Algoritmo SenoCoseno
2   Definir anguloGrados, anguloRadianes Como Real
3   Definir valorSeno, valorCoseno Como Real
4
5   Escribir "Ingrese el valor de un angulo en grados:"
6   Leer anguloGrados
7
8   // Las funciones trigonometricas de PSeInt trabajan en
   radianes
9   anguloRadianes <- anguloGrados * PI / 180
10
11  valorSeno <- sen(anguloRadianes)
12  valorCoseno <- cos(anguloRadianes)
13
14  Escribir "Seno de ", anguloGrados, " grados: ", valorSeno
15  Escribir "Coseno de ", anguloGrados, " grados: ", valorCoseno
16
17 FinAlgoritmo
```

Algoritmo 4: Seno y coseno de un ángulo ingresado en grados

Complejidad: $O(1)$. El algoritmo ejecuta una cantidad fija de operaciones, independiente de cualquier talla.

[↑ Volver al índice](#)

8.3. c) Hipotenusa de un triángulo rectángulo

Se ingresan por teclado los catetos de un triángulo rectángulo. Se desea hallar y mostrar su hipotenusa.

Por el teorema de Pitágoras, la hipotenusa se obtiene como:

$$h = \sqrt{c_1^2 + c_2^2}$$

Se incorpora la validación de ambos catetos, dado que una longitud debe ser estrictamente positiva para que el triángulo exista.

```
1 Algoritmo Hipotenusa
2   Definir cateto1, cateto2, hipotenusa Como Real
3
4   Repetir
5     Escribir "Ingrese la longitud del primer cateto:"
6     Leer cateto1
7     Si cateto1 <= 0 Entonces
8       Escribir "Error: la longitud debe ser mayor a 0."
9     FinSi
10  Hasta Que cateto1 > 0
11
12  Repetir
13    Escribir "Ingrese la longitud del segundo cateto:"
14    Leer cateto2
15    Si cateto2 <= 0 Entonces
16      Escribir "Error: la longitud debe ser mayor a 0."
17    FinSi
18  Hasta Que cateto2 > 0
19
20  hipotenusa <- rc(cateto1^2 + cateto2^2)
21
22  Escribir "La hipotenusa del triangulo rectangulo es: ",
23    hipotenusa
24 FinAlgoritmo
```

Algoritmo 5: Cálculo de la hipotenusa de un triángulo rectángulo

Complejidad: $O(1)$.

[↑ Volver al índice](#)

8.4. d) Cara de un dado mostrada en letras

Mostrar en letras el número de la cara de un dado obtenido al azar.

Se emplea `Aleatorio(1, 6)`, que devuelve un entero al azar entre ambos extremos inclusive, modelando el lanzamiento de un dado de seis caras. Para convertir ese número a letras se utiliza la selección múltiple `Segun...FinSegun`, más legible que una cadena de seis condicionales anidados. La rama `De Otro Modo` se incluye por buena práctica defensiva.

```
1 Algoritmo CaraDadoEnLetras
2   Definir cara Como Entero
3
4   // Aleatorio(1,6) genera un entero al azar entre 1 y 6
   inclusive
5   cara <- Aleatorio(1, 6)
6
7   Escribir "Se lanzo el dado y salio el numero ", cara
8
9   Segun cara Hacer
10      1:
11         Escribir "En letras: UNO"
12      2:
13         Escribir "En letras: DOS"
14      3:
15         Escribir "En letras: TRES"
16      4:
17         Escribir "En letras: CUATRO"
18      5:
19         Escribir "En letras: CINCO"
20      6:
21         Escribir "En letras: SEIS"
22      De Otro Modo:
23         Escribir "Valor fuera del rango valido de un dado."
24   FinSegun
25
26 FinAlgoritmo
```

Algoritmo 6: Cara de un dado al azar mostrada en letras

Complejidad: $O(1)$. La selección múltiple evalúa a lo sumo seis alternativas, cantidad fija e independiente de la entrada.

[↑ Volver al índice](#)

8.5. e) Sumatoria de 10 números generados al azar

Hallar la sumatoria de 10 números generados al azar. Los números deben comprender el siguiente rango de valores [1...20].

Se emplea un bucle **Para** de diez iteraciones —la cantidad es conocida de antemano, lo que hace preferible esta estructura frente a **Mientras**— y un acumulador inicializado en cero antes de ingresar al ciclo.

Cada número se genera con **Aleatorio(1, 20)**, respetando el rango solicitado por el enunciado. Se muestra cada valor obtenido además del total, para permitir la verificación del resultado.

```
1 Algoritmo SumatoriaDiezNumerosAzar
2   Definir i, numero, suma Como Entero
3
4   suma <- 0
5
6   Para i <- 1 Hasta 10 Con Paso 1 Hacer
7       // Aleatorio(1,20) genera un entero al azar en el rango
           [1..20]
8       numero <- Aleatorio(1, 20)
9       suma <- suma + numero
10      Escribir "Numero ", i, ": ", numero
11  FinPara
12
13  Escribir "-----"
14  Escribir "La sumatoria de los 10 numeros es: ", suma
15
16 FinAlgoritmo
```

Algoritmo 7:]Sumatoria de 10 números aleatorios en el rango [1..20]

Complejidad: $O(1)$, dado que la cantidad de iteraciones es una constante fijada por el enunciado. Si en lugar de diez números se solicitaran n , el orden sería $O(n)$.

[↑ Volver al índice](#)

8.6. f) Promedio de edades en una veterinaria

En una veterinaria se desea saber el promedio de edad de gatos y perros (por separados) que fueron asistidos durante un mes. En total se registraron 30 animales y la veterinaria solo atiende gatos y perros.

El problema requiere mantener **cuatro acumuladores** de manera simultánea: la suma de edades y el contador de animales, para cada una de las dos especies. El promedio de cada grupo se obtiene recién al finalizar el recorrido, dividiendo la suma correspondiente por su contador.

Se validan tanto el tipo de animal —que debe ser "G" o "P", las únicas especies que atiende la veterinaria— como la edad, que no puede ser negativa. Antes de calcular cada promedio se verifica además que el contador correspondiente sea mayor que cero, evitando una división por cero en el caso límite de que durante el mes se hubiese atendido una sola especie.

```
1 Algoritmo VeterinariaPromedioEdades
2   Definir i Como Entero
3   Definir tipoAnimal Como Caracter
4   Definir edad Como Real
5   Definir sumaGatos, sumaPerros Como Real
6   Definir cantGatos, cantPerros Como Entero
7   Definir promedioGatos, promedioPerros Como Real
8
9   sumaGatos <- 0
10  sumaPerros <- 0
11  cantGatos <- 0
12  cantPerros <- 0
13
14  Para i <- 1 Hasta 30 Con Paso 1 Hacer
15      Escribir "--- Animal Nro ", i, " ---"
16
17      Repetir
18          Escribir "Ingrese el tipo de animal (G = gato / P =
19              perro):"
20          Leer tipoAnimal
21          Si tipoAnimal <> "G" Y tipoAnimal <> "P" Entonces
22              Escribir "Error: solo se atienden gatos (G) o
23                  perros (P)."
```

```
27      Leer edad
28      Si edad < 0 Entonces
29          Escribir "Error: la edad no puede ser negativa."
30      FinSi
31      Hasta Que edad >= 0
32
33      Si tipoAnimal = "G" Entonces
34          sumaGatos <- sumaGatos + edad
35          cantGatos <- cantGatos + 1
36      SiNo
37          sumaPerros <- sumaPerros + edad
38          cantPerros <- cantPerros + 1
39      FinSi
40  FinPara
41
42  Escribir "==== RESUMEN DEL MES ====="
43
44  Si cantGatos > 0 Entonces
45      promedioGatos <- sumaGatos / cantGatos
46      Escribir "Gatos atendidos: ", cantGatos, " - Edad
47          promedio: ", promedioGatos
48  SiNo
49      Escribir "No se atendieron gatos durante el mes."
50  FinSi
51
52  Si cantPerros > 0 Entonces
53      promedioPerros <- sumaPerros / cantPerros
54      Escribir "Perros atendidos: ", cantPerros, " - Edad
55          promedio: ", promedioPerros
56  SiNo
57      Escribir "No se atendieron perros durante el mes."
58  FinSi
FinAlgoritmo
```

Algoritmo 8: Promedio de edad de gatos y perros atendidos en un mes

Complejidad: $O(1)$ con la talla fija de 30 animales establecida por el enunciado; $O(n)$ en el caso general de n registros, ya que cada animal se procesa exactamente una vez.

[↑ Volver al índice](#)

8.7. g) Factorial de un número

El factorial de un número entero se denota de la siguiente manera « $n!$ » y su resultado es $n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1$. Por ejemplo: $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$ siendo el resultado 120. Se pide desarrollar un programa que lee un valor N y determine su factorial.

Formalmente, el factorial se define de manera recursiva como $n! = n \cdot (n - 1)!$ para $n > 0$, con el caso base $0! = 1$.

La resolución iterativa emplea un acumulador **multiplicativo** inicializado en 1 —y no en 0, como sería el caso de un acumulador de sumas— que se multiplica sucesivamente por cada entero desde 2 hasta n . Se valida que el número ingresado no sea negativo, ya que el factorial no está definido para dichos valores; el caso $n = 0$ se resuelve naturalmente, porque el bucle no llega a ejecutarse y el acumulador conserva su valor inicial 1, que es precisamente $0!$. El acumulador se declara como **Real** y no como **Entero** debido al crecimiento explosivo de la función: $13!$ ya excede el rango de un entero de 32 bits.

```
1 Algoritmo FactorialIterativo
2   Definir n, i Como Entero
3   Definir factorial Como Real
4   Repetir
5       Escribir "Ingrese un numero entero N mayor o igual a 0:"
6       Leer n
7       Si n < 0 Entonces
8           Escribir "Error: no existe factorial de un negativo."
9       FinSi
10  Hasta Que n >= 0
11
12  factorial <- 1
13  // Si n = 0 el bucle no se ejecuta y queda 0! = 1
14  Para i <- 2 Hasta n Con Paso 1 Hacer
15      factorial <- factorial * i
16  FinPara
17
18  Escribir "El factorial de ", n, " es: ", factorial
19 FinAlgoritmo
```

Algoritmo 9: Cálculo del factorial de un número por método iterativo

Complejidad: $O(n)$, donde la talla n es el *valor* del número ingresado y no la cantidad de datos.

[↑ Volver al índice](#)

Conclusión

El desarrollo de este trabajo permitió integrar los dos planos que aborda la Unidad III. En el plano teórico, se establecieron las herramientas que permiten evaluar un algoritmo con independencia de la máquina donde se ejecute: la distinción entre el análisis a priori y el a posteriori, la definición de la talla del problema como parámetro de referencia, y la notación asintótica como lenguaje común para expresar tasas de crecimiento.

El punto 7 mostró con claridad que la diferencia entre un algoritmo lineal y uno cuadrático no reside en la cantidad de instrucciones escritas sino en su **estructura**: dos bucles secuenciales mantienen el orden lineal, mientras que los mismos dos bucles anidados lo elevan a cuadrático. Esta distinción, aparentemente menor en el código, determina la diferencia entre un programa que escala y uno que se vuelve inutilizable al crecer el volumen de datos.

En el plano práctico, los algoritmos del punto 8 —todos de complejidad constante o lineal— pusieron el énfasis en aspectos complementarios a la eficiencia asintótica pero igualmente determinantes para la calidad de una solución: la validación sistemática de los datos de entrada, la elección de la estructura de control adecuada a cada situación (**Para** cuando la cantidad de iteraciones es conocida, **Repetir** para la validación de ingresos, **Segun** para la selección múltiple), el tratamiento de los casos límite como 0! o la división por cero, y la selección correcta del tipo de dato según el rango de valores esperado.

[↑ Volver al índice](#)